VERITASCOPE

Insight beyond Headlines

Veritascope
Global News Analyzer using AI

Infosys Virtual Internship 6.0

Developed under the guidance of mentor Mrs. P Sathya

By Team 1

Sanika Sharma
Gunti Jahnavi
Gunda Vaishnavi
Repati Girish Kumar
Akhil Ahammed Shaik
S Devipriya

CONTENTS

1. Introduction
    1.1 Scope
        1.2 Objectives
2. System Requirements
        2.1 Software Requirements
        2.2 Python Libraries
3. Software Architecture
    3.1 Overview
    3.2 Module 1: User Authentication and Profile
    3.3 Module 2: Data Ingestion and Pre-processing
    3.4 Module 3: NLP Analysis and Topic Modeling
    3.5 Module 4: Trend and Insight Generation
    3.6 Module 5: Visualization and Reporting
    3.7 Module 6: Admin Dashboard
4. Data Flow Diagram
5. Database Schema
6. Source Code
7. Sample Output Forms and Reports
8. Conclusion
9. Future Scope
10. References

1. INTRODUCTION

1.1 Scope

In today's interconnected world, the sheer volume of online news articles and social media discussions makes it challenging for individuals, businesses, and policymakers to accurately identify emerging trends and gauge public sentiment across various sectors. This information overload can lead to missed opportunities, delayed responses to critical events, and a lack of understanding of the broader societal mood. The “Veritascope Global News Analyzer using AI” project aims to address this by developing an intelligent Natural Language Processing (NLP) system.

This system will analyze a diverse range of online text data to identify global or national emerging trends and determine public sentiment across key sectors such as technology, environment, and economy. By providing actionable insights derived from vast amounts of unstructured data, Veritascope Global News Analyzer using AI will empower users to make more informed decisions and stay ahead of the curve.

1.2 Objectives

The main objectives of this project can be outlined as follows:

- Real-time (Simulated) Trend Identification: Automatically detect and highlight emerging topics and discussions across global/national news and social media.
- Accurate Sentiment Analysis: Provide granular sentiment insights (positive, negative, neutral) towards specific topics, entities, or sectors.
- Data-Driven Insights: Deliver actionable intelligence for businesses, researchers, and policymakers by distilling complex text data into understandable trends.
- Scalable Data Processing: Architecture designed to handle and process large volumes of unstructured text data efficiently.
- Interactive Visualization: User-friendly dashboards to visualize trends, sentiment shifts, and key entities over time.
- Secure User Access: Robust user authentication for personalized access to insights.

2. SYSTEM REQUIREMENTS

2.1 Software Requirements

Operating System: Windows 8+ or Linux with a web browser

Database: MySQL Workbench CE 8.0

Programming Software: Python 3.11.9 (Python 3.1 or above; 3.11+ doesn't support some features)

NewsAPI.org API

Internet Connectivity

2.2 Python Libraries

Flask>=2.2,<3.0
python-dotenv>=1.0.0
requests>=2.28.0
mysql-connector-python>=8.0.30
pandas>=2.0.0
numpy>=1.24.0
nltk>=3.8.0
gensim>=4.2.0
scikit-learn>=1.2.0
scipy>=1.10.0
keybert>=0.7.0
sentence-transformers>=2.2.0
transformers>=4.30.0
torch>=1.13.0
PyJWT>=2.6.0
Werkzeug>=2.2.0

3. FUNCTIONAL DESCRIPTION

3.1 Overview

Veritascope Global News Analyzer using AI is a project that requires implementation of much different functionality. The main functions performed by this software can be divided into 6 modules:

1. **User Authentication and Profile**
   This module implements user registration and login using standard email and password combination. This is done in a secure way using JWT (JSON Web Token) security. In addition, a user profile is created for preferred language and areas of interest (e.g., 'Technology', 'Economy', ‘Environment’, etc) for a personalized experience.

2. **Data Ingestion & Preprocessing Module**
   News articles are automatically collected using free API from newsapi.org. These articles are formatted into a structured pandas dataframe and stored in database. Text cleaning, tokenization, and normalization are done on this stored data to make it ready for processing.

3. **NLP Analysis Module and Topic Modeling**
   This module identifies the main themes and topics within the collected text data. Sentiment analysis is done on the same data to determine the emotional tone (positive,negative, neutral) of text related to identified topics or entities.Named Entity Recognition is implemented to extract key entities like organizations, locations,
and people.

4. **Trend & Insight Generation Module**
   Time Series Analysis is performed on the accumulated data and analytics to track the evolution and emergence of topics and sentimentover time.Correlation analysis between topics and sentiment is also computed. Algorithm is developed to identify "spikes" or "dips" indicating emerging trends or significant
shift.
5. **Visualization & Reporting Module**
   Interactive dashboard is developed to display trends,sentiment charts, and topic distributions. The dashboard features the ability to generate summary reports.

6. **Admin Dashboard**

In order to facilitate administrative tasks such as management of data sources (e.g., adding/removing news feeds), monitoring of system performance and data quality, reviewing of analyzed results, a fully-fledged admin dashboard is also part of this project.

3.2 Module 1

User Authentication and Profile

This module implements user registration and login using JWT (JSON Web Security) for secure user authentication. For this PyJWT and Werkzeug libraries are used. The payload is encoded using Flask secret key with the help of PyJWT library function to create a token. This ensures the credentials are not tampered with. It also ensures that the database does not need to be queried each time credentials are needed. The payload consists of email, password and flask secret key. This can be seen in code snippet below:

```
token_payload = {
            'exp': datetime.utcnow() + timedelta(days=1),
            'iat': datetime.utcnow(),
            'sub': str(user_data['id']),
            'username': user_data.get('username', user_data['email']),
            'role': user_data.get('role', 'user')
        }
token = jwt.encode(token_payload, secret_key, algorithm='HS256')
```

The same is decoded when needed as:

```
data = jwt.decode(token, secret_key, algorithms=['HS256'])
```

The user data is stored in users table and users.py script manages various functions related to it:

- **connect_db()**
  This function connects to users table and also ensures creation of the users table if it doesn't already exist.

- **register_user(email, password)**
  This function accepts email and password entered by the user and checks the email against the users table. If email is not found, new user is registered and the email along with hashed password and createdAt is stored in users table.Werkzeug is used to hash the password:

```
hashed_password = generate_password_hash(password)
```

- **login_user(email, password, secret_key)**
  This function deals with user authentication. It creates a secure JWT payload if the entered credentials are validated to ensure safe and secure user experience.

Profile Management for users is also added where users can select their preferred language, set a username as well as select interested topics for a personalized dashboard. These details are stored in user_preferences table and user_profile.py scripts handles the data input and modifications.

- **connect_db()**
  This function establishes connection to user_preferences table and also takes care of table creation if the table doesn't already exist.

- **get_user_profile(user_id)**
  This function takes in user_id and returns all the stores preferences of the user.

- **update_user_profile(user_id, username, language, interests)**
  This function updates preferences when user changes them. The updated preferences are then stored to the user_preferences table along with updatedAt timestamp.

3.3 Module 2

Data Ingestion and Pre-processing

This module ingests data using API from newapi.org. The data is collected from top_headlines as:

```
url = f"https://newsapi.org/v2/top-
headlines?language=en&pageSize={int(num_articles)}&apiKey={NEWS_API_KE
Y}"
response = requests.get(url)
data = response.json()
 articles = data.get("articles", [])
```

The news articles thus collected contain:

- title
- source.name

- publishedAt
- url
- description
- content
- urlToImage

These are inserted correctly into the news table under title, source, publishedAt, url, description, content and imageurl. The publishedAt obtained as ISO 8601 formatted datetime is stripped to string so that it can be stored as DATETIME datatype in MySQL. The functions handling these functions are defined in fetch_news.py:

- **create_database()**
  This function handles the creation of the database news_db as well as creates table news required to store the articles.

- **connect_db()**
  This table returns a mysql connection object for the table news.

- **fetch_live_news(topic=None, num_articles=10)**
  10 articles on default are fetched by this function and it returns those articles as a list of lists.

- **convert_publishedAt(publishedAt_str)**
  This function converts the ISO 8601 formatted string obtained from newsAPI for publishedAt into a datetime object using strptime() from python datetime library. It returns this converted datetime.

  ```
  datetime.strptime(publishedAt_str, "%Y-%m %dT%H:%M:%SZ")
  ```

- **store_news(articles)**
  This function takes as argument the list of lists received from the newsAPI through fetch_live_news and parses them one by one. Each of articles is passed to insert_news to be inserted into the database.

- **insert_news(article)**
  This function checks if the article passed to it already exists in the database by comparing article's url. If it exists, function returns. Otherwise, the article details are inserted into news table after converting article's publishedAt to datetime using convert_publishedAt. The SQL query for the insertion operation is as follows:

```
INSERT INTO news (title, source, publishedAt, url, description, content,
imageurl) VALUES (%s, %s, %s, %s, %s, %s, %s)",
      (
        article.get("title"),
        article.get("source", {}).get("name"),
        published,
        article.get("url"),
        article.get("description"),
        article.get("content"),
        article.get("urlToImage"),
      )
```

- **fetch_and_store()**
  This function makes calls to fetch_live_news and stored the returned articles list in a variable, articles. Then it passed those articles as argument to the store_news function. Essentially, this function acts as themain function that fetches live news, passes the list of articles to store_news which in turn passed each article to be inserted to news table by insert_news. This way the entire script fetch_news.py can be run as:

```
if __name__ == "__main__":
   fetch_and_store()
```

Pre-processing of data involves cleaning, normalizing, and preparing the raw textual data fetched from news articles. This critical step ensures that all text is in a consistent and analyzable format before being fed into NLP models for keyword extraction, sentiment analysis, or topic modeling. This functionality is encapsulated in the text_preprocessing.py file.

This module utilizes several key libraries:

NLTK: Used to provide a comprehensive list of English "stopwords" (common words like 'the', 'is', 'in' that are filtered out).

spaCy: A powerful NLP library used for efficient tokenization (splitting text into words) and lemmatization (reducing words to their root form).

TextBlob: Used to perform spelling correction on the processed text.

re (Regular Expressions): Used for low-level cleaning tasks like removing URLs and numbers.

The functions and logic for this module are defined in the function preprocess_text(text). This is the only function in this module.

- **preprocess_text(text)**
  This is the main function of the module. It accepts a single string of raw text as input and returns a fully cleaned version. This function ensures all text is standardized.
  The pre-processing pipeline follows these steps:

  1. Check for Empty Text
  If the input text is empty or None, it returns an empty string immediately.

```
if not text:
    return ""
```

  2. Convert to Lowercase
  The entire text is converted to lowercase to ensure uniformity.

```
text = text.lower()
```

  3. Remove URLs
  Using regular expressions all web URLs are found and removed.

```
text = re.sub(r"http\S+|www\S+|https\S+", '', text, flags=re.MULTILINE)
```

  4. Remove Punctuation
  All punctuation marks from the text are stripped.

```
text = text.translate(str.maketrans('', '', string.punctuation))
```

  5. Remove Numbers
  All digits (0-9) are removed from the text using regular expressions.

```
text = re.sub(r'\d+', '', text)
```

  6. Tokenization (via spaCy)
  The cleaned text is processed by spaCy to create a doc object, which segments the text into individual tokens (words).

```
nlp = spacy.load("en_core_web_sm")
doc = nlp(text)
```

  7. Stopword Removal & Lemmatization

The function iterates through each token removing any token that is a stopword (from the NLTK list) or consists only of whitespace.For all remaining tokens, it finds the lemma (the root form of the word).It specifically removes pronoun lemmas (like '-PRON-') to focus on content-bearing words.The valid, lemmatized tokens are collected in a list.

```
tokens = []
   for token in doc:
      if token.text in stop_words or token.is_space:
         continue
      lemma = token.lemma_.strip()
      if lemma and lemma != '-PRON-':
         tokens.append(lemma)
```

8. Join Tokens: The list of cleaned tokens is joined back together into a single string, separated by spaces.

```
cleaned_text = " ".join(tokens)
```

9. Spell Correction: The resulting string is passed to TextBlob to correct common spelling mistakes.

```
corrected_text = str(TextBlob(cleaned_text).correct())
```

The function then returns the final, cleaned, lemmatized, and spell-checked string which can be called by other parts of the application, such as the search functionality in Veritascope.py and the analysis pipeline in topic_selection.py.

3.4 Module 3

NLP Analysis and Topic Modeling

Module 3 is responsible for the core Natural Language Processing tasks that enrich the raw news data. This includes extracting keywords for quick summaries, identifying named entities, analyzing sentiment, and categorizing articles using topic modeling.

Keyword Extraction

To provide users with a quick, at-a-glance summary of an article's content, a keyword extraction module is implemented in backend/keyword_extractor.py. This module runs as part of the data

ingestion pipeline in Veritascope.py. This module uses the KeyBERT library, which leverages transformer models (like BERT) to find the words and phrases in a document that are most representative of the full text.
The process is handled by the extract_and_store_keywords() function:

- **extract_and_store_keywords()**
  This function connects to the database and selects all articles from news table where corresponding the keywords column in keywords table is NULL or empty, ensuring it only processes new content. For each article, it creates a single text document by combining the title and description fields. It initializes the KeyBERT model and calls the kw_model.extract_keywords() function to extract keywords. It is configured to find the Top 3 most relevant terms (top_n=3). It uses a keyphrase_ngram_range=(1, 2), meaning the extracted "keywords" can be single words or two-word phrases. It uses a standard list of English stop_words to ignore generic terms.
  The code snipped is as follows:

```
kw_model = KeyBERT()
keywords_dict = {}
    for i, row in raw_news.iterrows():
        title = row['title'] or ""
        description = row['description'] or ""
        doc = (title + " " + description).strip()
        if not doc:
           continue
        keywords = kw_model.extract_keywords(doc,
keyphrase_ngram_range=(1, 2), stop_words='english', top_n=3)
keywords_dict[row['id']] = [kw for kw, _ in keywords]
```

  The resulting list of keywords is joined into a single, comma-separated string and stored in the keywords column of the keywords table for that article.

Topic Modeling

To automatically categorize news articles into broad themes, the application implements BERTopic, a state-of-the-art, transformer-based topic modeling technique. This process is managed by the backend/topic_selection.py script. This module leverages a SentenceTransformer model to understand the meaning of articles and clusters them into coherent topics. This modern approach provides more accurate and meaningful topic assignments than traditional methods.

This implementation is supported by a robust database schema featuring two dedicated tables:

- **topics:** Stores a permanent record for each topic, including its id, a human-readable name, and the keywords that define it.
- **article_topics_mapping**: A relational table that links each article_id to a topic_id and stores a relevance_score to indicate the confidence of the assignment.

To ensure the main application runs quickly, this file is designed with two distinct modes of operation:

1. Training Mode
   This mode is a slow, intensive process, run manually by a developer, which trains the BERTopic model on the entire article database and saves the model to disk.

2. Assigning Mode
   This mode is a fast, lightweight process, called by the Veritascope.py application, which loads the pre-trained model and uses it to categorize only the new articles.

This separation is crucial for performance, as it prevents the web server from freezing while re-training the model on every user request. The module saves and loads the model in the backend/models/ directory:

BERTopic_model
The model containing trained topics along with their keywords from the training data.

The functions in this topic_selection.py are:

- **get_stopwords()**
  This helper function prepares the list of stopwords required for the CountVectorizer component of the BERTopic pipeline. It loads the standard nltk English stopword list and extends it with an extensive custom_stop_words set. This custom list filters out generic news-related, time-based, and source-specific terms that do not contribute to meaningful topics. Once that custom_word_list is appended, it returns a list of all stopwords.

  ```
  nlp = spacy.load("en_core_web_sm")
  stop_words = set(stopwords.words('english'))

  stop_words.update(custom_stop_words)
  return list(stop_words)
  ```

- **preprocess_text_for_bert(text)**

This function performs minimal text cleaning to prepare raw text for the BERTopic pipeline. It converts text to lowercase, removes all URLs, punctuation, and numbers using regular expressions, and strips extra whitespace. This process intentionally avoids lemmatization, as transformer models derive better context from more natural text. The preprocessed text is returned as a string after joining them.

```
if not text:
        return ""
    text = text.strip().lower()
    text = re.sub(r"http\S+|www\S+|https\S+", '', text,
flags=re.MULTILINE)
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\d+', '', text)
    text = " ".join(text.split())
    return text
```

- **create_and_sync_topic_tables()**
  This function creates and maintains the database tables required for topic modeling. It creates the topics and article_topics_mapping tables if they do not exist yet. It then iterates through the manual_topic_labels dictionary (a hard-coded map of topic IDs to names defined by the developer) and synchronizes this dictionary with the topics table, ensuring the database always reflects the application's human-readable topic names.

```
for topic_id, name in manual_topic_labels.items():
        try:
            cursor.execute("""
                INSERT INTO topics (id, name, created_at, updated_at)
                VALUES (%s, %s, NOW(), NOW())
                ON DUPLICATE KEY UPDATE name = %s, updated_at =
NOW();""", (topic_id, name, name))
        except Exception as e:
            print(f"Error syncing topic {topic_id} ('{name}'): {e}")
```

- **train_models()**
  This is the main training function. It is a slow, heavy operation intended to be run manually from the command line to build and save the topic model. First it calls create_and_sync_topic_tables() to ensure the database schema is ready. Then it fetches all articles from the news table to build a robust training set and preprocesses all article texts using preprocess_text_for_bert.

```
create_and_sync_topic_tables()
conn = connect_db()
cursor = conn.cursor()

print("Fetching all articles for training...")
cursor.execute("SELECT id, title, description, content FROM news")
rows = cursor.fetchall()
cursor.close()
conn.close()

if not rows:
   print("No articles in database to train on.")
   return

print(f"Preprocessing {len(rows)} articles (minimal cleaning)...")
docs_text = []
for row in rows:
   title = row[1] or ""
   description = row[2] or ""
   content = row[3] or ""
   text = (title + " " + description + " " + content)
   docs_text.append(preprocess_text_for_bert(text))
```

After this, the SentenceTransformer("all-MiniLM-L6-v2") model is loaded for embeddings and a CountVectorizer (using get_stopwords()) to turn the embeddings into vectors.

```
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
stop_words = get_stopwords()
vectorizer_model = CountVectorizer(stop_words=stop_words)
```

In the next step, the BERTopic model is initialized with these components and a min_topic_size to control topic granularity.

```
topic_model = BERTopic(
      embedding_model=embedding_model,
      vectorizer_model=vectorizer_model,
      language="english",
      verbose=True,
      min_topic_size=13
   )
```

Finally the scripts trains the model by calling topic_model.fit_transform() on the article texts and saves the fully trained model to the backend/models/BERTopic_model directory. The

keywords column in the topics table is updated using the results from topic_model.get_topic_info(), allowing topic keywords to be reviewed in the database.

```
try:
    topics, probabilities = topic_model.fit_transform(docs_text)
  except Exception as e:
    print(f"Error during BERTopic training: {e}")
    return

  print("Saving model...")
  topic_model.save(BERTOPIC_MODEL_PATH)

  print("\nTraining Complete!")

  print("\nTopics Detected:")
  topic_info = topic_model.get_topic_info()
  print(topic_info)

print("Updating 'topics' table with new keywords...")
  conn = connect_db()
  cursor = conn.cursor()
  for row in topic_info.to_dict('records'):
    topic_id = int(row['Topic'])
    keywords_str = ", ".join(row['Representation'])
    cursor.execute("""
      UPDATE topics SET keywords = %s, updated_at = NOW()
      WHERE id = %s""", (keywords_str, topic_id))
  conn.commit()
  conn.close()
  print("Topic keywords updated.")
```

Once training is complete, the developer can review the terminal output and update the manual_topic_labels dictionary if the discovered topics do not align with the defined names.

- **assign_topics()**
  This is the fast assigninginference) function, which is called by Veritascope.py to categorize new articles as they are fetched. First, it calls create_and_sync_topic_tables() to ensure the database is ready. Then it loads the pre-trained BERTopic_model from the backend/models/ directory. Next step is fetching only the new articles by performing a LEFT JOIN query to find articles in news that do not have an entry in the article_topics_mapping table.

```
cursor.execute("""
    SELECT n.id, n.title, n.description, n.content
    FROM news n
    LEFT JOIN article_topics_mapping at ON n.id = at.article_id
    WHERE at.article_id IS NULL;
  """)
rows = cursor.fetchall()
```

In the final step, the text of these new articles is preprocessed and passed to topic_model.transform() to efficiently get the predicted topic_ids and probabilities (relevance scores) for the new batch.

```
texts_to_assign = []
  doc_ids = []
  for row in rows:
    doc_id = row[0]
    title = row[1] or ""
    description = row[2] or ""
    content = row[3] or ""
    text = (title + " " + description + " " + content)
    texts_to_assign.append(preprocess_text_for_bert(text))
    doc_ids.append(doc_id)

  if not texts_to_assign:
    print("No processable text found in new articles.")
    cursor.close()
    conn.close()
    return

  try:
    # Get both the predicted topic ID and the probability matrix
    topic_ids, probabilities = topic_model.transform(texts_to_assign)
    # Get the relevance score (the probability of the assigned topic)
    relevance_scores = list(probabilities)

  except Exception as e:
    print(f"Error transforming new articles: {e}")
    cursor.close()
    conn.close()
    return
```

By iterating through the results, this script inserts a new row into the article_topics_mapping table for each article, linking the article_id to its new topic_id and storing the relevance_score.

```
# Loop through results and update DB
    updates = 0
    for i in range(len(doc_ids)):
        doc_id = doc_ids[i]
        topic_id = int(topic_ids[i])
        score = float(relevance_scores[i])

        try:
            cursor.execute("""
                INSERT INTO article_topics_mapping (article_id, topic_id,
relevance_score, assigned_at)
                VALUES (%s, %s, %s, NOW())""", (doc_id, topic_id, score))
            updates += 1
        except Exception as e:
            if "Duplicate entry" not in str(e):
                print(f"Error updating article {doc_id}: {e}")
```

Sentiment Analysis

In order to determine the emotional tone (positive, negative, or neutral) of each news article, sentiment analysis is done. This functionality is encapsulated in backend/sentiment.py. The module uses a pre-trained model from the Hugging Face transformers library to perform the analysis. The results are then stored in a dedicated sentiments table in the database for quick retrieval and aggregation. The various functions in this script are:

- **connect_db()**
  This function establishes a connection to the MySQL database and ensures the sentiments table exists. If it does not exist, a query is excuted to create the sentiments table. This table stores the article_id, float values for positive, neutral, and negative scores, and a text field for the final overall sentiment.

- **analyze_sentiment(text)**
  This function analyzes a single string of text and returns a structured dictionary of its sentiment. This is done using the sentiment_analyzer = pipeline("sentiment-analysis") object, which loads a powerful, pre-trained Hugging Face model. Once the input text is run through this model, a primary label (POSITIVE or NEGATIVE) and a confidence score is returned. This result is processed into a standardized four-part dictionary, containing positive, neutral, negative, and overall keys.

```
result = sentiment_analyzer(text)
    label = result[0]['label']
    score = result[0]['score']
```

```
    if label == 'POSITIVE':
        return {
            'positive': int(round(score * 100)),
            'neutral': int(round((1-score) * 100)),
            'negative': 0,
            'overall': 'Positive'
        }
    else:
        return {
            'positive': 0,
            'neutral': int(round((1-score) * 100)),
            'negative': int(round(score * 100)),
            'overall': 'Negative'
        }
```

- **save_sentiment(article_id, sentiment_dict)**
  This function stores the analysis results from analyze_sentiment into the database.It connects to the database and executes an INSERT query to add a new row to the sentiments table, linking the article_id to the provided positive, neutral, negative, and overall values.

```
conn = connect_db()
    cursor = conn.cursor()
    query = '''INSERT INTO sentiments (article_id, positive, neutral,
negative, overall)
             VALUES (%s, %s, %s, %s, %s)'''
    cursor.execute(query, (
        article_id,
        float(sentiment_dict['positive']),
        float(sentiment_dict['neutral']),
        float(sentiment_dict['negative']),
        str(sentiment_dict['overall'])
    ))
    conn.commit()
    conn.close()
```

- **analyze_and_save_sentiments()**
  This is the main function, designed to be run by Veritascope.py. It finds all new articles and populates their sentiment data. It does this by connecting to the database and performing a LEFT JOIN query to find all articles in the news table that do not have a corresponding entry in the sentiments table.

```
conn = connect_db()
```

```
cursor = conn.cursor(dictionary=True)
cursor.execute('''SELECT n.id
                  FROM news n
                  LEFT JOIN sentiments s ON n.id = s.article_id
                  WHERE s.article_id IS NULL;''')
article_rows = cursor.fetchall()
```

If no new articles are found, it prints a message and exits.If new articles are found, it iterates through each one, fetches its title and description, and combines them into a single text string. Then it calls analyze_sentiment()on this text to get the results and calls save_sentiment() to store these results in the database.

```
for row in article_rows:
    article_id_int = row['id']
    cursor.execute('''SELECT title, description FROM news where id =
%s''', (article_id_int,))
    article = cursor.fetchone()
    if not article:
       continue
    title = article.get('title') or ""
    description = article.get('description') or ""
    text = title + " " + description
    if not text.strip():
       continue
    sentiment = analyze_sentiment(text)
    save_sentiment(article_id_int, sentiment)
```

Named Entity Recognition

This component is responsible for identifying and extracting key entities, such as people, organizations, and locations, from the news articles. This functionality is defined in backend/ner.py. The module uses the powerful spaCy library, loading its "en_core_web_sm" model to perform the entity extraction. The extracted entities are stored in a dedicated entities table. The various functions that help achieve this task are:

- **connect_db()**
  This function establishes a connection to the MySQL database and ensures the entities table exists. If it doesn't exist, a query for the entities table creation is executed. The table includes columns for article_id, name (the entity text), type (its label), and confidence. Crucially, it creates an INDEX(article_id) to speed up future queries, such as finding articles that have already been processed.

- **extract_entities(text)**
  The role of this function is to find and categorize named entities within a given text string, ensuring no duplicates. It processes the input text with the globally loaded spaCy model.It then filters entities, keeping only those with allowed_labels (PERSON, ORG, GPE, LOC).To ensure efficiency and prevent redundant data, it converts the list of found entities into a set of (text, label) tuples. This automatically removes all duplicates. A list of unique entity dictionaries is returned.

```
doc = nlp(text)
allowed_labels = {'PERSON', 'ORG', 'GPE', 'LOC'}
unique_ents = list(set([(ent.text, ent.label_) for ent in doc.ents if ent.label_ in allowed_labels]))
return [{'text': text, 'label': label} for text, label in unique_ents]
```

- **save_entities(article_id, entities_list)**
  This function stores a batch of extracted entities for a single article into the database in a highly efficient manner. First, it prepares a list of tuples, where each tuple contains the article_id, entity text, label, and confidence.

```
rows_to_insert = []
for ent in entities_list:
    rows_to_insert.append((
        article_id,
        ent['text'],
        ent['label'],
        float(ent.get('confidence', 1.0))
    ))
```

  It then uses the highly efficient cursor.executemany() method to insert all entities for that article in a single database transaction, which is much faster than inserting them one by one.This operation is wrapped in a try...finally block to ensure the database connection is closed properly, even if an error occurs.

```
try:
    if rows_to_insert:
        cursor.executemany(query, rows_to_insert)
        conn.commit()
except Exception as e:
    print(f"Error saving entities for article {article_id}: {e}")
finally:
    conn.close()
```

- **analyze_and_save_entities()**
  This is the main function, designed to be run as a script from Veritascope.py. It finds all new articles, analyzes them for entities, and saves the results. New articles are found by connecting to database and performing a LEFT JOIN query to find all articles in the news table that do not have a corresponding entry in the entities table.

```
conn = connect_db()
cursor = conn.cursor(dictionary=True)
cursor.execute('''
    SELECT n.id, n.title, n.description, n.content
    FROM news n
    LEFT JOIN entities e ON n.id = e.article_id
    WHERE e.article_id IS NULL;
  ''')
articles_to_process = cursor.fetchall()
```

  If no new articles are found, it prints a message and exits.
  If new articles are found, it iterates through each one and combines the article's title and description to create the analysis text. Then, extract_entities()is called to get the unique entity list and save_entities()is called to save this batch of entities to the database.

```
for article in articles_to_process:
    article_id = article['id']
    title = article['title'] or ""
    description = article['description'] or ""
    text = title + " " + description

    if not text.strip():
       continue

    print(f"Extracting entities for article {article_id}...")

    entities_list = extract_entities(text)
    save_entities(article_id, entities_list)
```

3.5 Module 4

Trend and Insight Generation

The **Trend and Insight Generation** module is a core component of Veritascope that automatically analyzes news articles to identify emerging trends, track sentiment patterns, and provide actionable insights to users. This system processes real-time news data to deliver meaningful analytics through interactive visualizations and trend detection algorithms.

Core Components

Trend Detection Engine:

File: trend_detector.py

Purpose: Identifies trending topics and articles in real-time

Key Features:

- Analyzes article frequency and engagement patterns
- Detects emerging topics using keyword clustering
- Ranks trends based on velocity and volume
- Categorizes trends into predefined categories (Politics, Technology, Sports, etc.)

Key Methods:

- get_daily_trends(): Returns current trending topics and articles
- analyze_trend_velocity(): Measures how quickly topics are growing
- categorize_trends(): Groups trends into meaningful categories

Trend Dashboard Overview

Purpose:
To provide a real-time, at-a-glance view of global news trends, allowing users to quickly understand what topics and articles are currently most prominent.

Key Components & Features:

Real-Time Status:
Clearly displays the last update time (e.g., Last updated: 2025-11-11 23:12), emphasizing the live nature of the data.

Trending Topics Section:
Format: Presents topics as clustered keywords in a table format.

Function: Allows users to infer the subject of a trend without a predefined label, showcasing the engine's ability to detect emerging narratives.

TREND ANALYSIS ROUTES

These routes provide a complete trend analysis system with a main dashboard view and dedicated API endpoints. The main /trends route serves a full HTML page displaying trending content, while the separate /trending-articles and /trending-topics endpoints deliver JSON data for dynamic frontend applications. All routes are protected with authentication and leverage the TrendDetector engine to analyze and serve real-time trending news data.

```
@app.route("/trends")
@token_required
def trends():
    """Main trends page showing trending topics and articles"""
    detector = TrendDetector()
    trends_data = detector.get_daily_trends()

    return render_template(
        "trends.html",
        trends_data=trends_data,
        user=g.username,
        now=datetime.now(),
        role=g.role
    )

@app.route("/trending-articles")
@token_required
def trending_articles():
    """API endpoint to get trending articles"""
    detector = TrendDetector()
    trends_data = detector.get_daily_trends()

    return jsonify({
        'trending_articles': trends_data['trending_articles'],
        'top_keywords': trends_data['keywords']
    })

@app.route("/trending-topics")
@token_required
def trending_topics():
    """API endpoint to get trending topics"""
```

```
detector = TrendDetector()
trends_data = detector.get_daily_trends()

return jsonify({
    'topics': trends_data['topics'],
    'categories': trends_data['trend_categories']
})
```

3.6 Module 5

Visualization and Reporting

Module 5 is the analytical engine of Veritascope, responsible for transforming raw historical data into actionable insights and predictive intelligence. This module performs **time-series analysis**, **trend detection**, and **forecasting** to help users understand not only what is happening now, but what is likely to happen next. The module leverages the accumulated historical data from the database and applies statistical techniques to identify emerging patterns in news volume, sentiment shifts, and topic frequency over time.

Overview

This module consists of two primary components:

1. **Historical Analytics** - Analyzing past 7-day, 90 days and 30-day trends in news volume, sentiment distribution, and topic popularity

2. **Predictive Analytics** - Forecasting future trends using time-series models to predict news volume, sentiment scores, and topic frequency for upcoming days

The functionality is implemented in **backend/analytics_utils.py**, with results being served to the frontend dashboard via dedicated API endpoints.

Technologies Used:

- pandas (data processing and aggregation)
- numpy (numerical calculations)
- scikit-learn (forecasting and statistical modeling)
- scipy (advanced stats, smoothing)
- Prophet (time-series forecasting model)

Database Tables Involved:

- **news:** for publishedAt timestamps and article counts

- **sentiments**: for sentiment scores over time
- **article_topics_mapping:** for topic distribution tracking
- **topics**: for topic metadata

Functional Breakdown

Historical Trend Analysis :

The historical analytics component provides users with insight into recent patterns by aggregating data over two key windows: 7 days and 30 days.

Functions in analytics_utils.py:

- **fetch_daily_counts:**

This foundational function queries the database to calculate the daily count of records for any specified table (e.g., news). It ensures only non-null, non-empty entries are counted over the defined historical period. The output is used to plot raw volume data and to create time-series input for forecasting models.

```
def fetch_daily_counts(table, column, start_days_ago=90):
    conn = connect_db()
    cursor = conn.cursor()
    query = f"""
        SELECT DATE(publishedAt) as day, COUNT(*) as cnt
        FROM {table}
        WHERE publishedAt>= CURDATE() - INTERVAL %s DAY
        AND {column} IS NOT NULL AND {column} != ''
        GROUP BY day
        ORDER BY day ASC;
      """
    cursor.execute(query, (start_days_ago,))
    days, counts = [], []
    for day, cnt in cursor.fetchall():
      days.append(str(day))
      counts.append(cnt)
    cursor.close(); conn.close()
    return days, counts
```

Output: A list of dictionaries containing date and count for each day, which is used to plot time-series charts on the dashboard.

- **get_news_volume_timeseries**

This function leverages fetch_daily_counts to retrieve article volumes and immediately formats the data into a **Pandas DataFrame**. The resulting DataFrame adheres to the required ds (date) and y (count) schema, making it ready for direct input into the Prophet forecasting model.

```
def get_news_volume_timeseries(days=90):
    days_list, counts = fetch_daily_counts('news', 'id',
    start_days_ago=days)
    return pd.DataFrame({'ds': days_list, 'y': counts}), days_list, counts
```

Output: Daily averages of sentiment scores, enabling visualization of sentiment shifts over time.

- **def get_sentiment_timeseries(days=90):**

This function analyzes the categorical sentiment labels (positive, neutral, negative) from the sentiments table. It groups these labels by day to show the count of articles for each sentiment over time. This data is the source for the main sentiment trend chart**.**

```
def get_sentiment_timeseries(days=90):
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT DATE(n.publishedAt), s.overall, COUNT(*)
        FROM news n
        JOIN sentiments s ON n.id = s.article_id
        WHERE n.publishedAt>= CURDATE() - INTERVAL %s DAY
        GROUP BY DATE(n.publishedAt), s.overall
        ORDER BY DATE(n.publishedAt) ASC;
      """, (days,))
    days_set = set()
    by_day = {}
    for day, sentiment, cnt in cursor.fetchall():
        day = str(day)
        days_set.add(day)
        if day not in by_day:
            by_day[day] = {'positive':0, 'neutral':0, 'negative':0}
        s = sentiment.lower()
        if s in by_day[day]:
```

```
            by_day[day][s] = cnt
    days_list = sorted(list(days_set))
    pos = [by_day[d]['positive'] for d in days_list]
    neu = [by_day[d]['neutral'] for d in days_list]
    neg = [by_day[d]['negative'] for d in days_list]
    cursor.close(); conn.close()
    return days_list, pos, neu, neg
```

Output: A dataset showing daily topic distribution, which can be visualized as stacked area charts or line graphs.

- **def get_topic_timeseries(topic_id, days=90):**

This function tracks the historical relevance of a single, specified topic. It counts the number of articles tagged with a given topic_id for each day of the historical window. This time-series data can used to visualized as a trend line or used for topic-specific forecasting.

```
def get_topic_timeseries(topic_id, days=90):
    conn = connect_db()
    cursor = conn.cursor()
     cursor.execute("""
          SELECT DATE(n.publishedAt), COUNT(*)
          FROM article_topics_mappingatm
          JOIN news n ON atm.article_id = n.id
          WHERE n.publishedAt>= CURDATE() - INTERVAL %s DAY
            AND atm.topic_id = %s
          GROUP BY DATE(n.publishedAt)
          ORDER BY DATE(n.publishedAt) ASC;
       """, (days, topic_id))
    days, counts = [], []
    for day, cnt in cursor.fetchall():
    days.append(str(day))
    counts.append(cnt)
    cursor.close(); conn.close()
    return pd.DataFrame({'ds': days, 'y': counts}), days, counts
```

Output: DataFrame and two lists: days (date strings) and counts (articles per day) for the specified topic.

Predictive Analytics

This set of functions implements forecasting capabilities using the**Prophet** library, predicting future trends for up to 7 days ahead.

- **def forecast_timeseries(df, periods=7):**

This is the core engine for prediction, using the **Prophet** model. It takes a historical time-series DataFrame, trains the model, and generates future values. The function is designed to be reusable for forecasting news volume, sentiment counts, or topic frequency.
Python

```
def forecast_timeseries(df, periods=7):
   if df.dropna().shape[0] < 2:
      return [], []
   m = Prophet()
   m.fit(df)
   future = m.make_future_dataframe(periods=periods)
   forecast = m.predict(future)
   fcast_dates = forecast['ds'][-periods:].dt.strftime('%Y-%m-%d').tolist()
   fcast_values = forecast['yhat'][-periods:].round(2).tolist()
    return fcast_dates, fcast_values
```

Output : Two lists: forecasted dates and predicted values (yhat) for the specified number of future periods (e.g., 7 days).

- **def get_news_volume_timeseries_and_forecast(days=90, predict_days=7):**

This function combines the historical data retrieval (get_news_volume_timeseries) with the prediction logic (forecast_timeseries). It provides a single output structure containing both the actual historical volume and the predicted future volume for seamless dashboard visualization.

```
def get_news_volume_timeseries_and_forecast(days=90, predict_days=7):
   df, history_dates, history_counts = get_news_volume_timeseries(days)
   fcast_dates, fcast_values = forecast_timeseries(df,
periods=predict_days)
   return {
      "history_dates": history_dates,
```

```
    "history_counts": history_counts,
    "fcast_dates": fcast_dates,
    "fcast_values": fcast_values
  }
```

- **Output** : A dictionary containing historical article counts per day and predicted counts for the next 7 days (history_dates, history_counts, fcast_dates, fcast_values).

- **def <u>get_sentiment_timeseries_and_forecast(days=90, predict_days=7):</u>**

This function handles the comprehensive sentiment forecast by calling get_sentiment_timeseries to get historical data. It then applies the forecast_timeseries model separately to the historical counts for each category (positive, neutral, negative) to predict their future counts.

```
def get_sentiment_timeseries_and_forecast(days=90, predict_days=7):
    days_list, pos, neu, neg = get_sentiment_timeseries(days)
    df_pos = pd.DataFrame({'ds': days_list, 'y': pos})
    df_neu = pd.DataFrame({'ds': days_list, 'y': neu})
    df_neg = pd.DataFrame({'ds': days_list, 'y': neg})
    pos_fcast_dates, pos_fcast = forecast_timeseries(df_pos,
periods=predict_days)
    neu_fcast_dates, neu_fcast = forecast_timeseries(df_neu,
periods=predict_days)
    neg_fcast_dates, neg_fcast = forecast_timeseries(df_neg,
periods=predict_days)
    return {
      "days": days_list,
      "pos": pos, "neu": neu, "neg": neg,
      "pos_fcast_dates": pos_fcast_dates, "pos_fcast": pos_fcast,
      "neu_fcast_dates": neu_fcast_dates, "neu_fcast": neu_fcast,
      "neg_fcast_dates": neg_fcast_dates, "neg_fcast": neg_fcast
    }
```

Output: A dictionary with daily historical sentiment counts (positive, neutral, negative), plus forecasted counts per sentiment for the next 7 days.

- **def get_topic_timeseries_and_forecast(topic_id, days=90, predict_days=7):**

This function streamlines the process of predicting a specific topic's future prominence. It fetches the topic's historical counts, passes the data to the Prophet model via forecast_timeseries, and returns a combined data structure with both historical and forecast counts.

```
def get_topic_timeseries_and_forecast(topic_id, days=90,
predict_days=7):
   df, history_dates, history_counts = get_topic_timeseries(topic_id, days)
   fcast_dates, fcast_values = forecast_timeseries(df,
periods=predict_days)
   return {
      "topic_id": topic_id,
      "history_dates": history_dates,
      "history_counts": history_counts,
      "fcast_dates": fcast_dates,
      "fcast_values": fcast_values
   }
```

Output : A dictionary with topic id, historical daily counts and dates, and predicted counts/dates for the next 7 days.

.

Utility and Summary Functions

These functions provide critical data for topic lists, summary cards, and the alternative numerical sentiment views on the dashboard.

- **get_existing_topics(min_articles=20):**

This utility function queries the database to find all topics that have been assigned to at least a minimum number of articles. This ensures the system only tracks and displays relevant topics that have established sufficient data volume for meaningful analysis.

```
def get_existing_topics(min_articles=20):
   conn = connect_db()
   cursor = conn.cursor()
cursor.execute("""
      SELECT t.id, t.name
      FROM topics t
      JOIN article_topics_mappingatm ON atm.topic_id = t.id
```

```
        GROUP BY t.id, t.name
        HAVING COUNT(atm.article_id) > %s
        ORDER BY t.id ASC;
    """, (min_articles,))
    topics = [{'id': row[0], 'name': row[1]} for row in cursor.fetchall()]
  cursor.close()
  conn.close()
     return topics
```

Output : List of dictionaries, each with a topic's id and name, filtered by minimum article assignment.

.

- **def get_top_topics_from_db(days=7, limit=5):**

This function determines the current most popular topics within a short, recent window (e.g., 7 days). It counts article assignments and returns the top limit topics, providing a quick, actionable view of what is trending now.

```
def get_top_topics_from_db(days=7, limit=5):
   conn = connect_db()
   cursor = conn.cursor()
   cursor.execute("""
      SELECT atm.topic_id, t.name, COUNT(*) as count
      FROM article_topics_mappingatm
      JOIN topics t ON atm.topic_id = t.id
      WHERE atm.assigned_at>= CURDATE() - INTERVAL %s DAY
      GROUP BY atm.topic_id, t.name
      ORDER BY count DESC
      LIMIT %s;
   """, (days, limit))
   topics = [{'label': name, 'count': count} for _, name, count in
cursor.fetchall()]
   cursor.close()
   conn.close()
   return topics
```

Output : List of top topics (label and count) for the recent days window, limited to the specified number.

- **def get_sentiment_distribution_numerical():**

This function calculates the overall sentiment distribution based on the numerical scores stored in the sentiments table (as opposed to categorical labels). It returns a clean percentage breakdown of positive, neutral, and negative sentiment across all data.

```
def get_sentiment_distribution_numerical():
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("SELECT SUM(positive), SUM(neutral), SUM(negative)
FROM sentiments")
    pos, neu, neg = cursor.fetchone()
    total = pos + neu + neg
    if total == 0:
        stats = {'positive': 0, 'neutral': 0, 'negative': 0}
    else:
        stats = {
            'positive': round(pos * 100 / total),
            'neutral': round(neu * 100 / total),
            'negative': round(neg * 100 / total)
        }
    cursor.close()
    conn.close()
     return stats
```

Output: Dictionary with percentage distribution of positive, neutral, and negative sentiment scores (e.g. {'positive': 42, 'neutral': 6, 'negative': 52}).

- **def get_sentiment_numerical_trend_by_day(days=90):**

This function is an alternative to get_sentiment_trend_by_day, calculating the daily sentiment percentage trend based on the numerical scores. By calculating the percentage of each sentiment per day, it standardizes the trend data independent of daily article volume fluctuations.

```
def get_sentiment_numerical_trend_by_day(days=90):
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute("""
        SELECT DATE(n.publishedAt),
               COUNT(*),
               SUM(s.positive), SUM(s.neutral), SUM(s.negative)
        FROM news n
```

```
        JOIN sentiments s ON n.id = s.article_id
        WHERE n.publishedAt>= DATE_SUB(CURDATE(), INTERVAL %s
DAY)
        GROUP BY DATE(n.publishedAt)
        ORDER BY DATE(n.publishedAt) ASC;
    """, (days,))
    days_list, pos_list, neu_list, neg_list = [], [], [], []
    for day, total, pos, neu, neg in cursor.fetchall():
        days_list.append(str(day))
        pos_pct = int(round((pos / total) if total else 0))
        neu_pct = int(round((neu / total) if total else 0))
        neg_pct = int(round((neg / total) if total else 0))
        pos_list.append(pos_pct)
        neu_list.append(neu_pct)
        neg_list.append(neg_pct)
    cursor.close()
    conn.close()
    return {
        'days': days_list,
        'positive': pos_list,
        'neutral': neu_list,
        'negative': neg_list
    }
```

Output : Dictionary with lists of days and daily sentiment percentages (positive, neutral, negative).

- **def get_sentiment_percentage_forecast(days=90, predict_days=7):**

This sophisticated function combines historical sentiment percentage calculation with forecasting. It first calculates the daily sentiment percentages using numerical scores and then applies the Prophet model separately to forecast the future percentage trend for positive, neutral, and negative sentiment.

```
def get_sentiment_percentage_forecast(days=90, predict_days=7):
    conn = connect_db()
    cursor = conn.cursor()
cursor.execute("""
        SELECT DATE(n.publishedAt), COUNT(*),
            SUM(s.positive), SUM(s.neutral), SUM(s.negative)
        FROM news n
        JOIN sentiments s ON n.id = s.article_id
```

```
        WHERE n.publishedAt>= DATE_SUB(CURDATE(), INTERVAL %s
DAY)
        GROUP BY DATE(n.publishedAt)
        ORDER BY DATE(n.publishedAt) ASC;
    """, (days,))
days_list, pos_list, neu_list, neg_list = [], [], [], []
    for day, total, pos, neu, neg in cursor.fetchall():
days_list.append(str(day))
pos_pct = int(round((pos / total) if total else 0))
neu_pct = int(round((neu / total) if total else 0))
neg_pct = int(round((neg / total) if total else 0))
pos_list.append(pos_pct)
neu_list.append(neu_pct)
neg_list.append(neg_pct)
cursor.close()
conn.close()

df_pos = pd.DataFrame({'ds': days_list, 'y': pos_list})
df_neu = pd.DataFrame({'ds': days_list, 'y': neu_list})
df_neg = pd.DataFrame({'ds': days_list, 'y': neg_list})

pos_fcast_dates, pos_fcast = forecast_timeseries(df_pos,
periods=predict_days)
neu_fcast_dates, neu_fcast = forecast_timeseries(df_neu,
periods=predict_days)
neg_fcast_dates, neg_fcast = forecast_timeseries(df_neg,
periods=predict_days)

    return {
        "days": days_list,
        "pos": pos_list, "neu": neu_list, "neg": neg_list,
        "pos_fcast_dates": pos_fcast_dates, "pos_fcast": pos_fcast,
        "neu_fcast_dates": neu_fcast_dates, "neu_fcast": neu_fcast,
        "neg_fcast_dates": neg_fcast_dates, "neg_fcast": neg_fcast
    }
```

Output : Dictionary with historical daily sentiment percentages and forecasted percentages for each sentiment category for the next 7 days.

- **def get_sentiment_stats_from_db(days=90):**

This function calculates the overall sentiment distribution based on the categorical labels (overall column) in the sentiments table. It

aggregates the total counts for all time and converts these into rounded percentages for the sentiment pie chart visualization on the dashboard.

```
def get_sentiment_stats_from_db(days=90):
    conn = connect_db()
    cursor = conn.cursor()
    cursor.execute('''
        SELECT overall, COUNT(*) FROM sentiments GROUP BY overall;
    ''')
    stats = {'positive': 0, 'neutral': 0, 'negative': 0}
    total = 0
    for label, count in cursor.fetchall():
        l = label.lower()
        if l == 'positive':
            stats['positive'] = count
        elif l == 'neutral':
            stats['neutral'] = count
        elif l == 'negative':
            stats['negative'] = count
        total += count
    cursor.close()
    conn.close()
    # Optionally convert counts to percentages
    if total > 0:
        for k in stats:
            stats[k] = int(round(stats[k] * 100 / total))
    return stats
```

Output : Dictionary with rounded percentage distribution of sentiment (positive, neutral, negative) based on categorical labels.

- **def get_sentiment_trend_by_day(days=90):**

This function is essential for the main dashboard line chart. It groups the categorical sentiment labels by day, creating three distinct lists (positive, neutral, negative counts) that can be plotted as separate time-series lines showing the daily trend in article volume for each sentiment.

```
def get_sentiment_trend_by_day(days=90):
    conn = connect_db()
    cursor = conn.cursor()
```

```
    cursor.execute('''
      SELECT DATE(n.publishedAt) as day, s.overall, COUNT(*) as cnt
      FROM news n
      JOIN sentiments s ON n.id = s.article_id
      WHERE n.publishedAt>= DATE_SUB(CURDATE(), INTERVAL %s
DAY)
      GROUP BY day, s.overall
      ORDER BY day ASC;
    ''', (days,))
    days_list = []
    data_dict = {}
    for day, sentiment, cnt in cursor.fetchall():
      if day not in days_list:
        days_list.append(day)
      if day not in data_dict:
        data_dict[day] = {'positive': 0, 'neutral': 0, 'negative': 0}
      s = sentiment.lower()
      if s in data_dict[day]:
        data_dict[day][s] = cnt

    # Generate result lists
    positive = [data_dict[d]['positive'] for d in days_list]
    neutral = [data_dict[d]['neutral'] for d in days_list]
    negative = [data_dict[d]['negative'] for d in days_list]
    cursor.close()
    conn.close()
    return {'days': days_list, 'positive': positive, 'neutral': neutral,
'negative': negative}
```

Output: Dictionary with lists of dates and daily counts per sentiment (positive, neutral, negative).

Integration with Dashboard

The analytics functions are exposed through Flask API endpoints in Veritascope.py to seamlessly deliver data to the frontend:

```
@app.route('/api/analytics', methods=['GET'])
defget_analytics():
  period = request.args.get('period', '7')  # 7 or 30 days
  days = int(period)

analytics_data = {
    'historical': {
```

```
        'news_volume': get_news_volume_trends(days=days),
        'sentiment': get_sentiment_trends(days=days),
        'topics': get_topic_frequency_trends(days=days)
    },
    'forecast': {
        'news_volume': forecast_news_volume(days_ahead=3),
        'sentiment': forecast_sentiment_trends(days_ahead=3),
        'topics': forecast_topic_frequency(days_ahead=3)
    },
    'trends': detect_trends(metric='volume', threshold=1.5)
}

returnjsonify(analytics_data)
```

The frontend then renders:

- **Line charts** showing past 7/30 days of actual data
- **Dashed forecast lines** extending into the future
- **Highlighted trend markers** indicating significant spikes or dips

3.7 Module 6

Admin Dashboard

The Veritascope Administrator Dashboard serves as the central command center for system administrators, providing a high-level overview of platform health, user activity, and content volume. It is designed for at-a-glance monitoring and efficient management of all system operations.
The dashboard is segmented into two primary sections:

1. **System Overview:** A summary of key platform metrics and status.
2. **User Management:** A detailed interface for administering user accounts.

The interface displays real-time data, enabling administrators to ensure system stability and manage user access effectively.

User Management Module

This module provides a comprehensive interface for overseeing all registered user accounts. Administrators can view, edit, and manage user profiles and their associated permissions.
Functional Description:
The "All Users" screen presents a paginated or scrollable list of every user registered on the platform. For each user, the following details are displayed:

- **Name:** The user's display name.
- **Email:** The user's registered email address.

- **Role:** The assigned permission level (admin or user).
- **Registered:** The date the account was created.
- **Actions:** Interactive buttons to Edit or Delete the user account.

Key Features:

- **Role-Based Access Control (RBAC):** Clearly distinguishes between Administrator accounts, which have full system access, and Regular Users, who have standard application privileges.
- **Account Lifecycle Management:** Allows administrators to modify user details or deactivate accounts as necessary.
- **Data Integrity:** The interface reflects the total number of accounts, providing immediate context for user base growth.

Database Tables Involved:

- users: Stores user credentials, roles, and registration dates.

Integration:
The frontend fetches user data from a backend API endpoint (e.g., GET /api/admin/users) and sends update or deletion requests (e.g., POST /api/admin/users/<user_id>/edit, DELETE /api/admin/users/<user_id>) to manage accounts seamlessly.

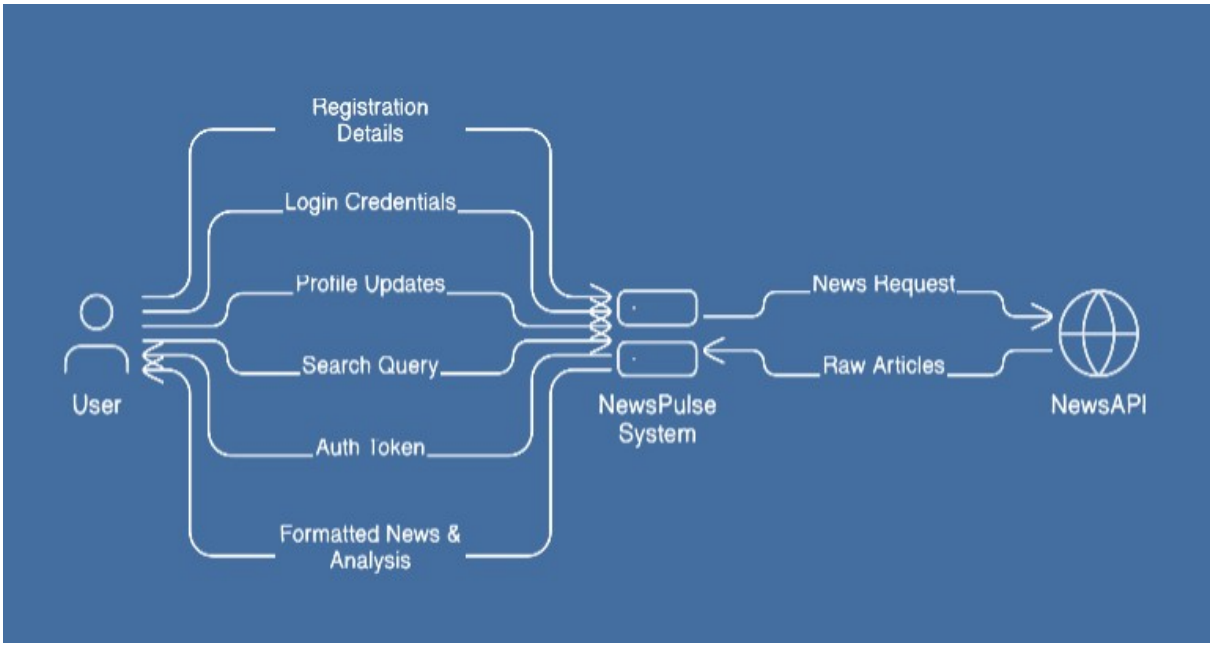
System Status & Core Metrics

This component provides a real-time snapshot of the platform's operational health and the scale of its processed data.
Displayed Metrics:

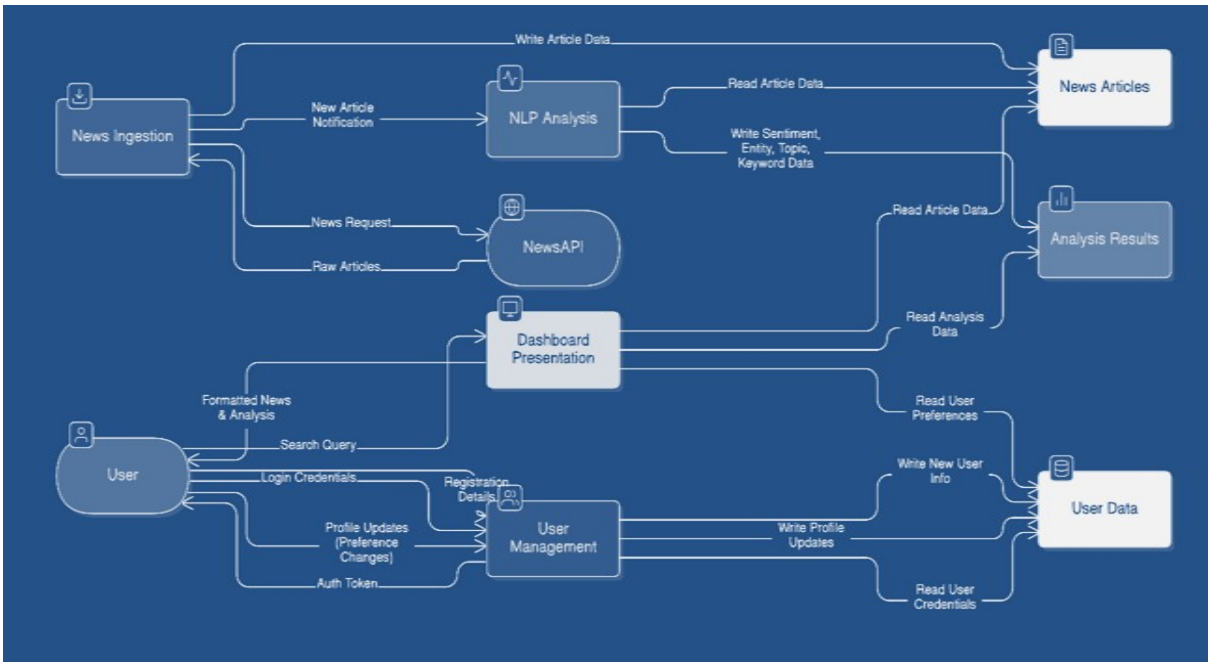
- **System Status:** Indicates whether all backend services (e.g., News Fetcher, NLP Engine, Database) are Online and operational.
- **Total Articles:** The cumulative count of news articles processed and stored in the database.
- **Total Keywords:** The number of unique keywords extracted from all articles.
- **Total Topics:** The number of distinct topics identified and tracked by the system.
- **Avg. Response Time:** The average latency of API responses, a key performance indicator for user experience.

4. DATA FLOW DIAGRAM

Level 0 DFD

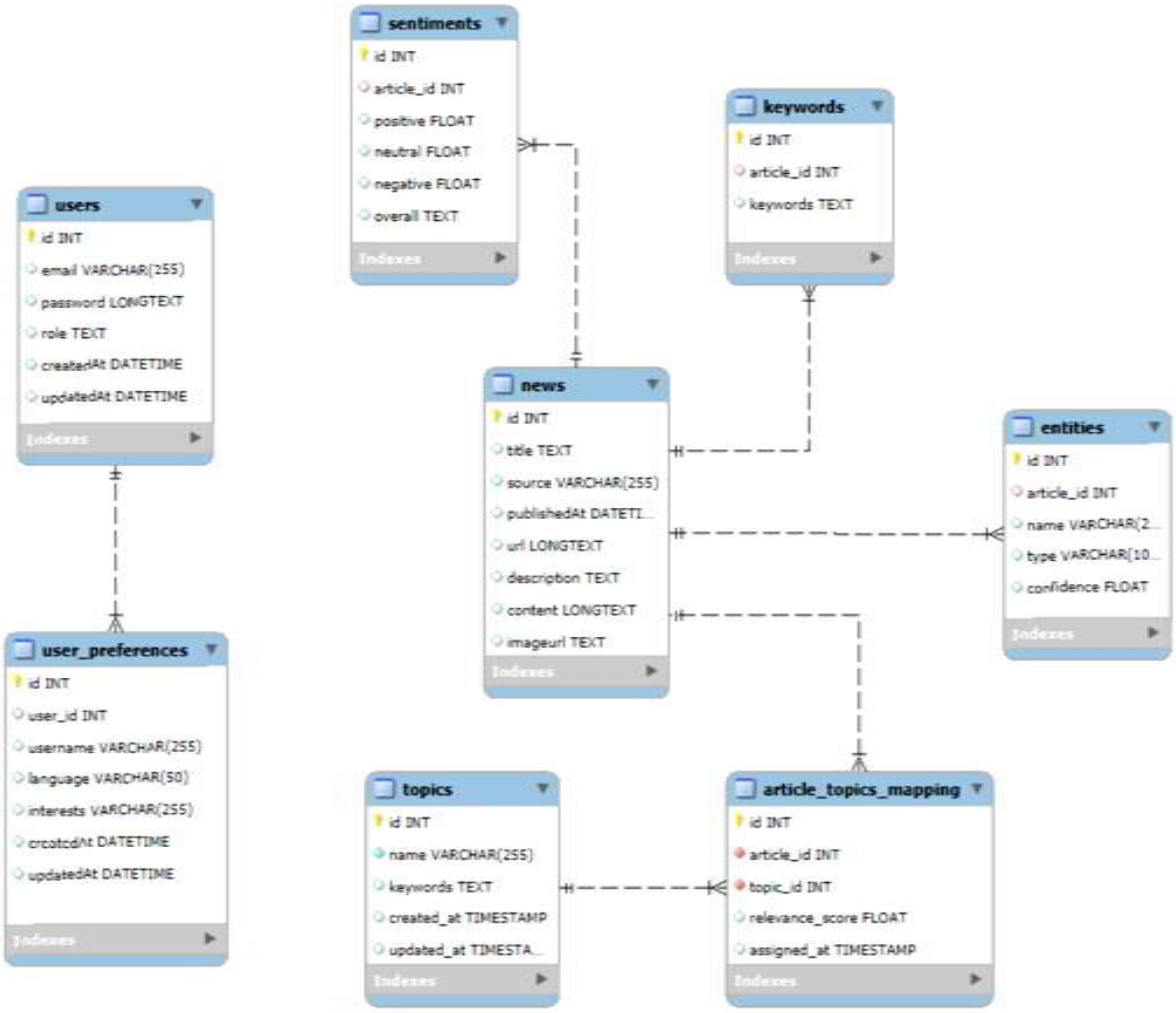


Level 1 DFD



5. DATABASE SCHEMA

The overall structure of the database newsdb can be depicted in the Entity Relationship (ER) diagram in Crow's foot notation as given below.



From this database ER diagram, we can create database schema for the individual tables. These tables along with their SQL DDL queries are listed below one by one.

news table

Schema

id	title	source	publishedAt	url	description	content	Imageurl

Primary Key: id

SQL query

```
CREATE TABLE IF NOT EXISTS news (
        id INT AUTO_INCREMENT PRIMARY KEY,
        title TEXT,
        source VARCHAR(255),
        publishedAt DATETIME,
        url LONGTEXT,
        description TEXT,
        content LONGTEXT,
        imageurl TEXT,
    );
```

users table

Schema

id	email	password	role	createdAt	updatedAt

Primary Key: id

SQL query

```
CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        email VARCHAR(255) UNIQUE,
        password LONGTEXT,
        role TEXT,
        createdAt DATETIME,
        updatedAt DATETIME);
```

user_preferences table

Schema

id	user_id	username	language	interests	createdAt	publishedAt

Primary Key: id
Foreign Key: user_id (references users)

SQL query

```
CREATE TABLE IF NOT EXISTS user_preferences (
        id INT AUTO_INCREMENT PRIMARY KEY,
        user_id INT UNIQUE,
        username VARCHAR(255),
        language VARCHAR(50),
        interests VARCHAR(255),
        createdAt DATETIME,
        updatedAt DATETIME,
        FOREIGN KEY (user_id) REFERENCES users(id));
```

keywords table

Schema

Id	article_id	keywords

Primary Key: id
Foreign Key: article_id (references news)

SQL query

```
CREATE TABLE IF NOT EXISTS keywords(
                id INT AUTO_INCREMENT PRIMARY KEY,
                article_id INT UNIQUE,
                keywords TEXT,
                FOREIGN KEY(article_id) REFERENCES news(id)
            );
```

topics table

Schema

id	name	keywords	createdAt	updatedAt

Primary Key: id

SQL query

```
CREATE TABLE IF NOT EXISTS topics (
        id INT PRIMARY KEY,
```

```
        name VARCHAR(255) NOT NULL,
        keywords TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON
UPDATE CURRENT_TIMESTAMP,
        UNIQUE(name)
    );
```

article_topics_mapping table

Schema

id	article_id	topic_id	relevance_score	assigned_at

Primary Key: id
Foreign Key: article_id (references news), topic_id (references topics)

SQL query

```
CREATE TABLE IF NOT EXISTS article_topics_mapping (
        id INT AUTO_INCREMENT PRIMARY KEY,
        article_id INT NOT NULL,
        topic_id INT NOT NULL,
        relevance_score FLOAT,
        assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (article_id) REFERENCES news(id),
        FOREIGN KEY (topic_id) REFERENCES topics(id),
        UNIQUE KEY (article_id, topic_id)
    );
```

sentiments table

Schema

id	article_id	positive	neutral	negative	overall

Primary Key: id
Foreign Key: article_id (references news)

SQL query

```
CREATE TABLE IF NOT EXISTS sentiments (
            id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    article_id INT,
    positive FLOAT,
    neutral FLOAT,
    negative FLOAT,
    overall TEXT,
    FOREIGN KEY (article_id) REFERENCES news(id)
);
```

entities table

Schema

id	article_id	Name	type	confidence

Primary Key: id
Foreign Key: article_id (references news)

SQL query

```
CREATE TABLE IF NOT EXISTS entities (
    id INT AUTO_INCREMENT PRIMARY KEY,
    article_id INT,
    name VARCHAR(255),
    type VARCHAR(100),
    confidence FLOAT,
    FOREIGN KEY (article_id) REFERENCES news(id),
    INDEX(article_id)
);
```

6. SOURCE CODE

The source code for this project is available at the github link:

Veritascope Global Trend Analyzer using AI
(https://github.com/S-Devipriya/Veritascope-Global-News-Trend-Analyzer-Team-1)

The structure of our project is given in the following diagram:

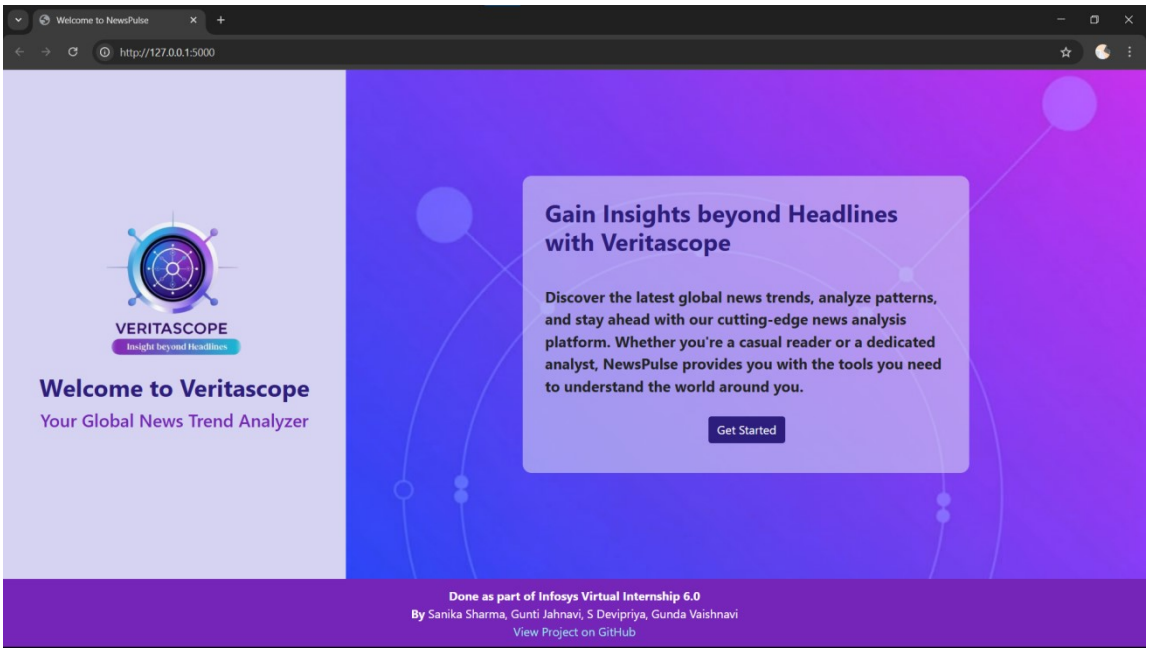
```
• backend
    • models
        • BERTopic_model
    • fetch_news.py
    • keyword_extractor.py
    • ner.py
    • Veritascope.py
    • sentiment.py
    • text_preprocessing.py
    • topic_selection.py
    • trend_detector.py
    • user_profile.py
    • users.py
```
```
• static
    • images
        • background.png
        • icon.png
        • mic.jpg
        • speaker
```
```
• templates
    • admin_dashboard.html
    • article_detail.html
    • dashboard.html
    • home.html
    • login.html
    • profile.html
    • register.html
    • trends.html
```
```
```

• .example.env
```
```
• .gitignore
```
```
• LICENCE
```
```
• README.md
```
```
• requirements.txt
```
```

7. SAMPLE OUTFUT FORMS AND REPORTS

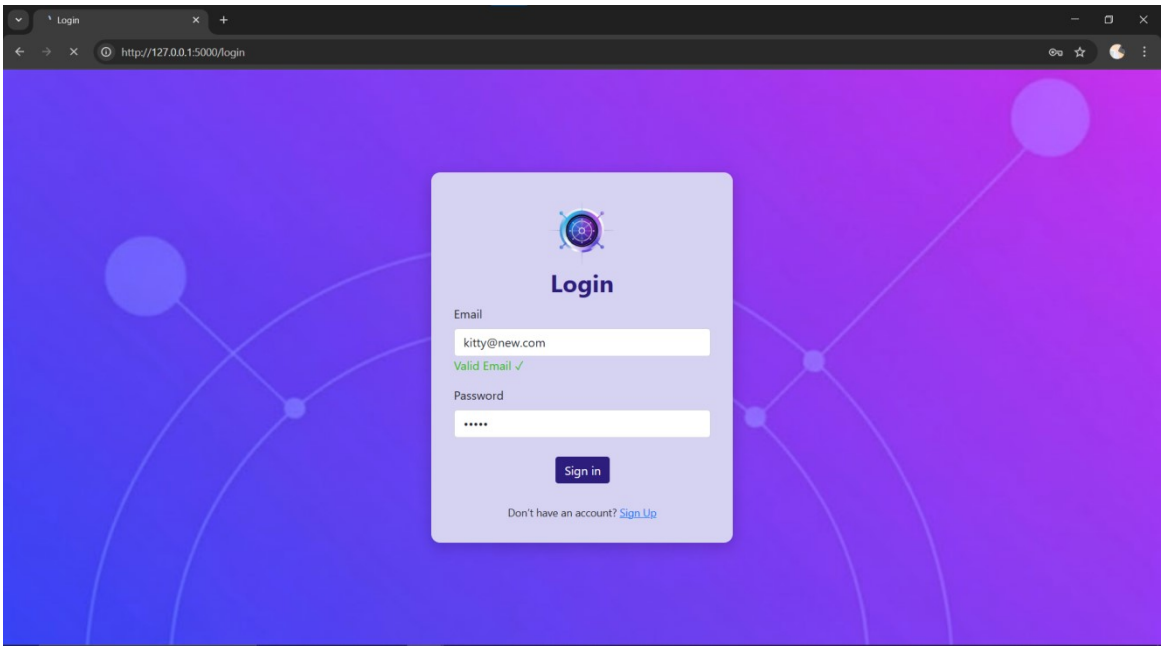
Welcome Screen

This is the welcome screen to our app. It shows the name of app and project details.



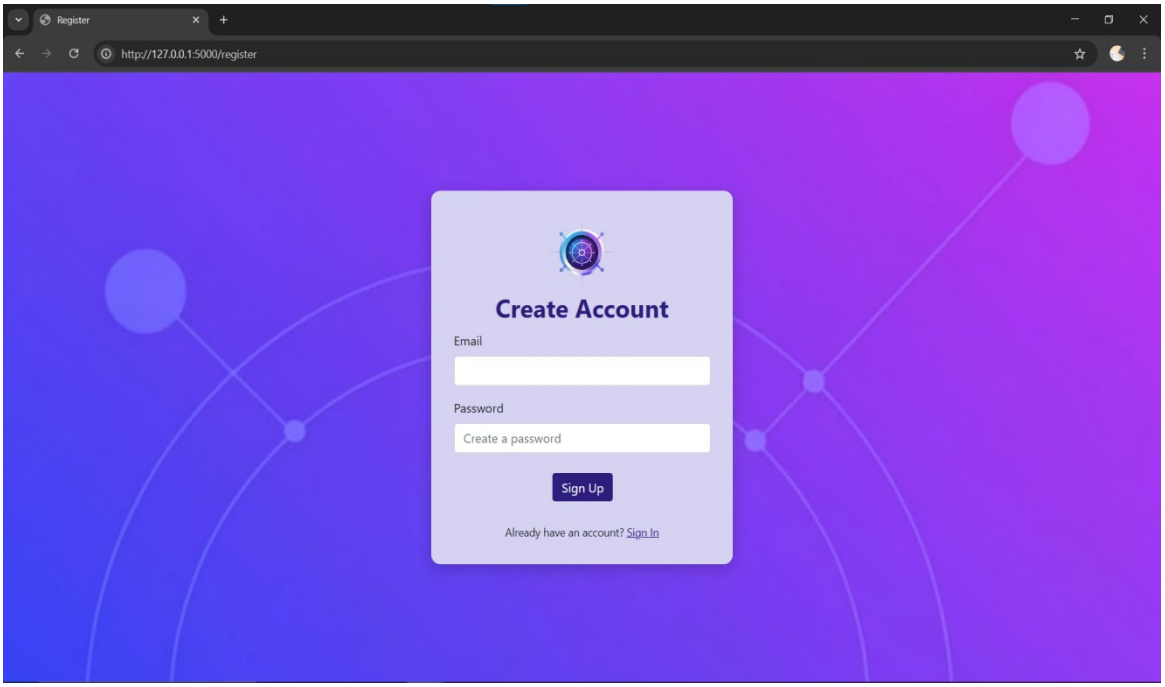
Login Form

The login form for users is shown below. This form is used for both user and admin authentication.



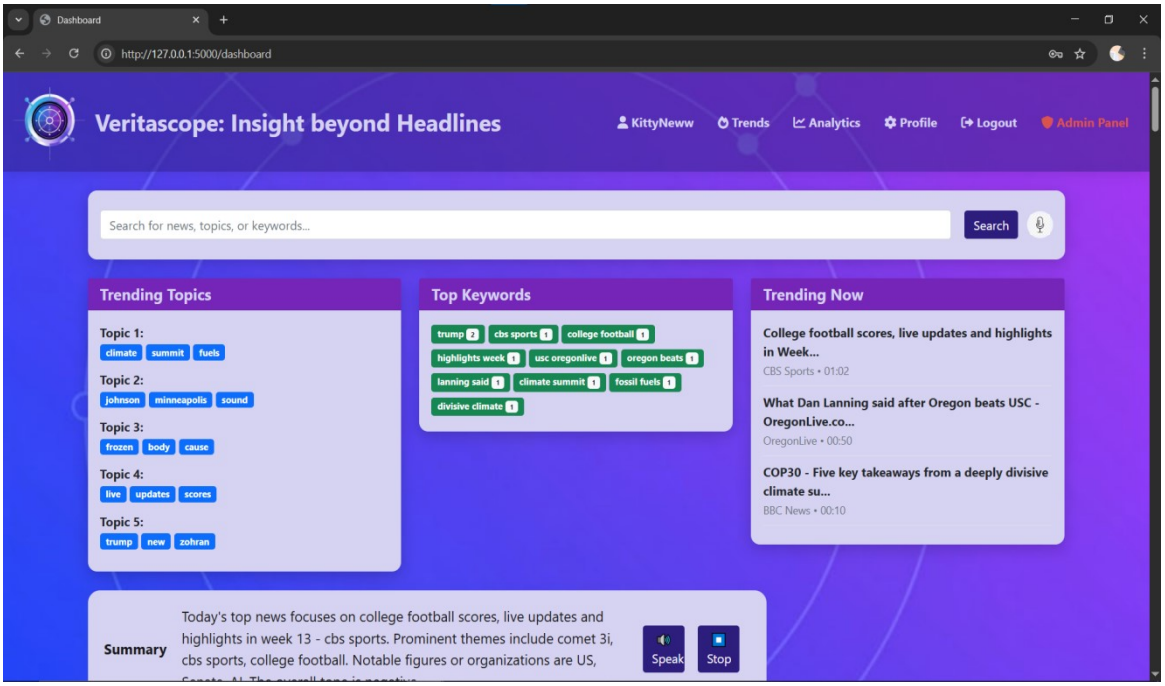
Registration Form

This form is for user registration and account creation.

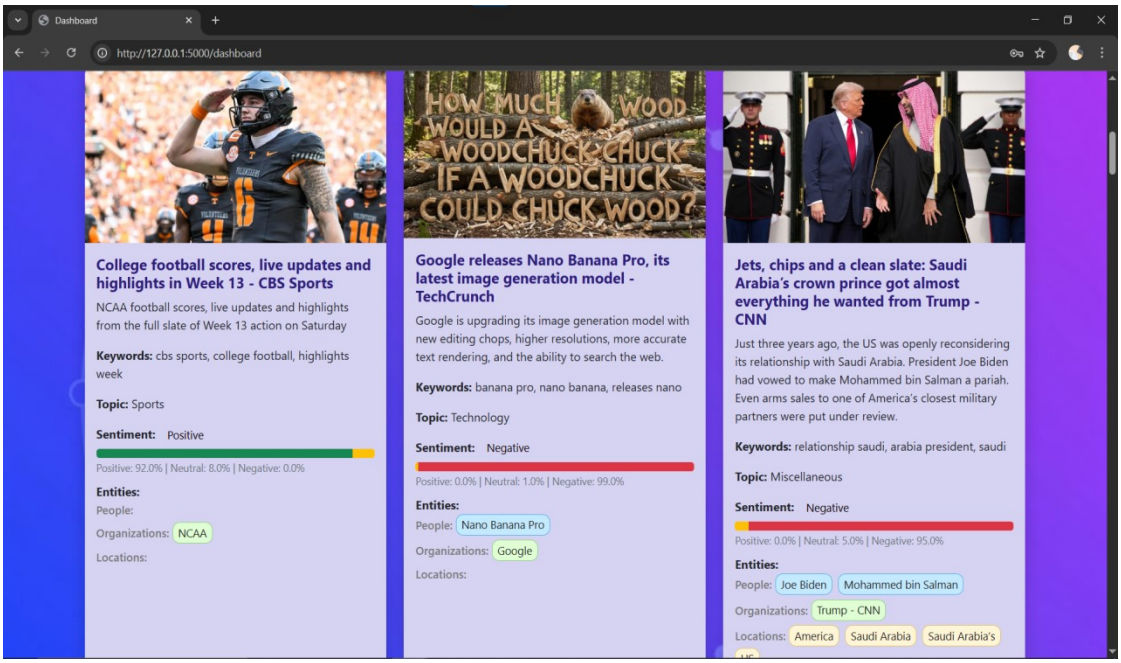


Dashboard

The main dashboard for users is as follows. It has a navigation bar that connects to various other pages.

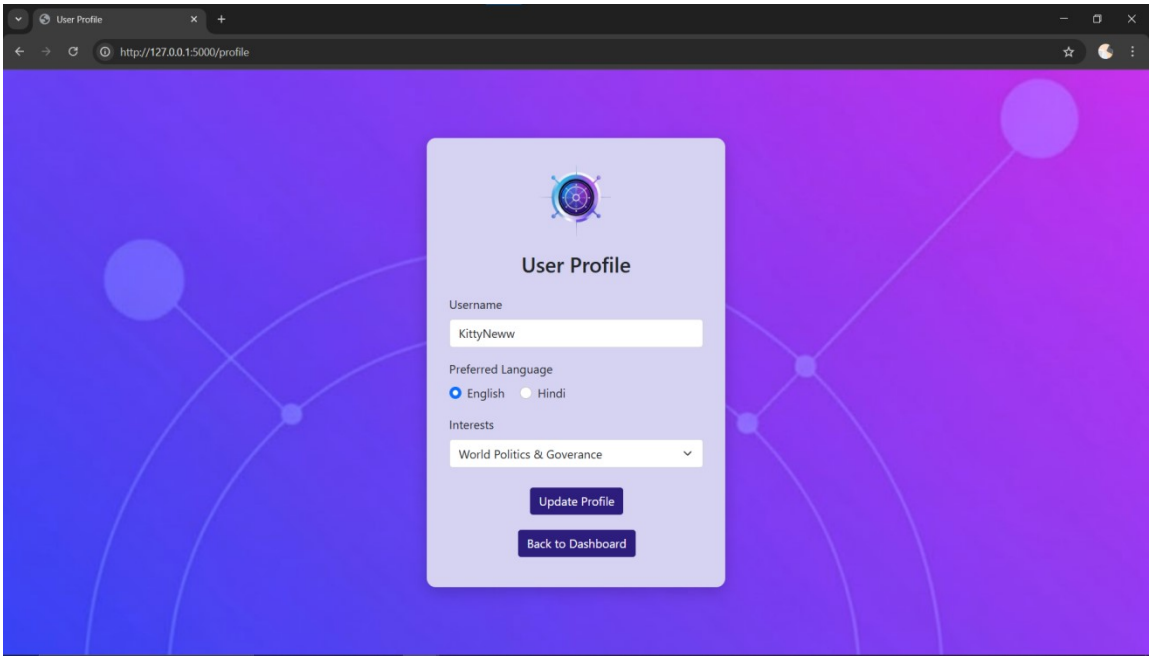


The dashboard also features scrollable news articles fetched using query given in search bar or sorted by latest published date.



Profile Management

Users can manage their profile in this page.



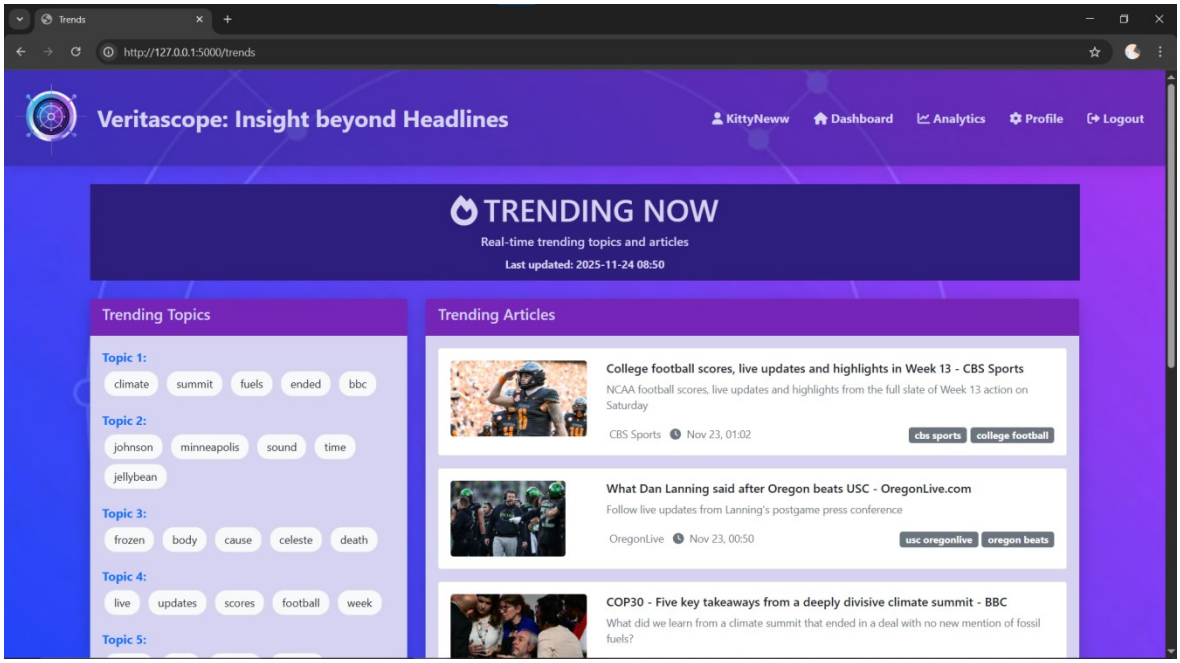
Topic, Sentiment and Entity Report

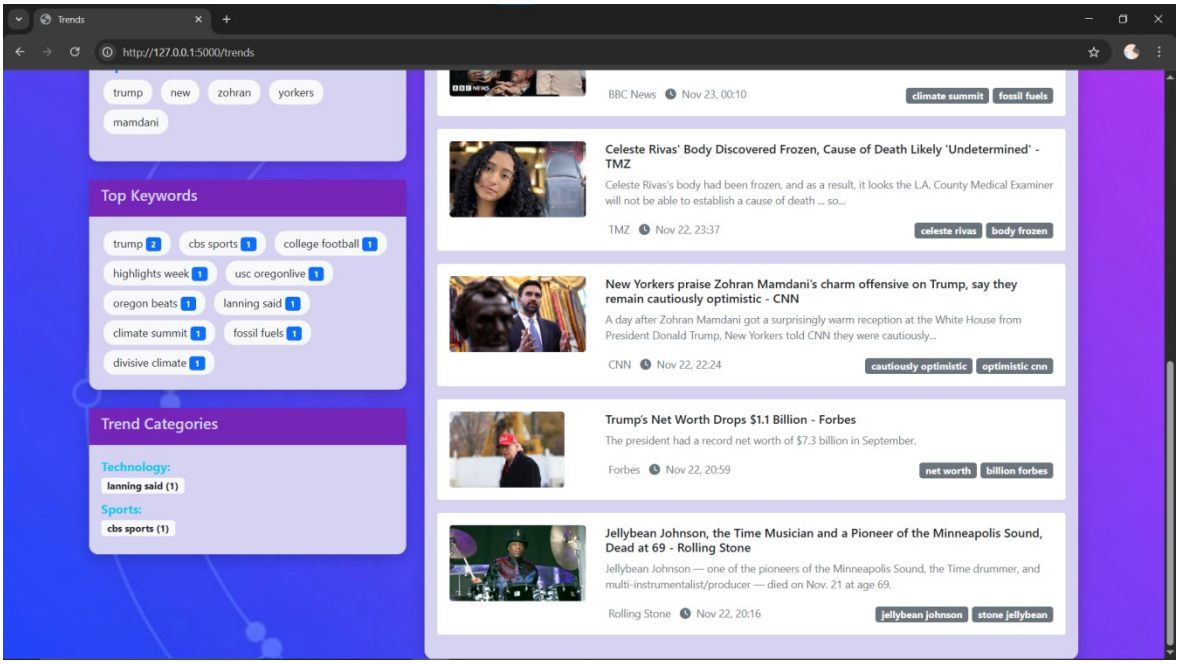
Each article in dashboard features a detailed analysis of its sentiment shown via a sentiment meter, its topic cluster and entities identified in that article. An example article card is shown below.



Trends Page

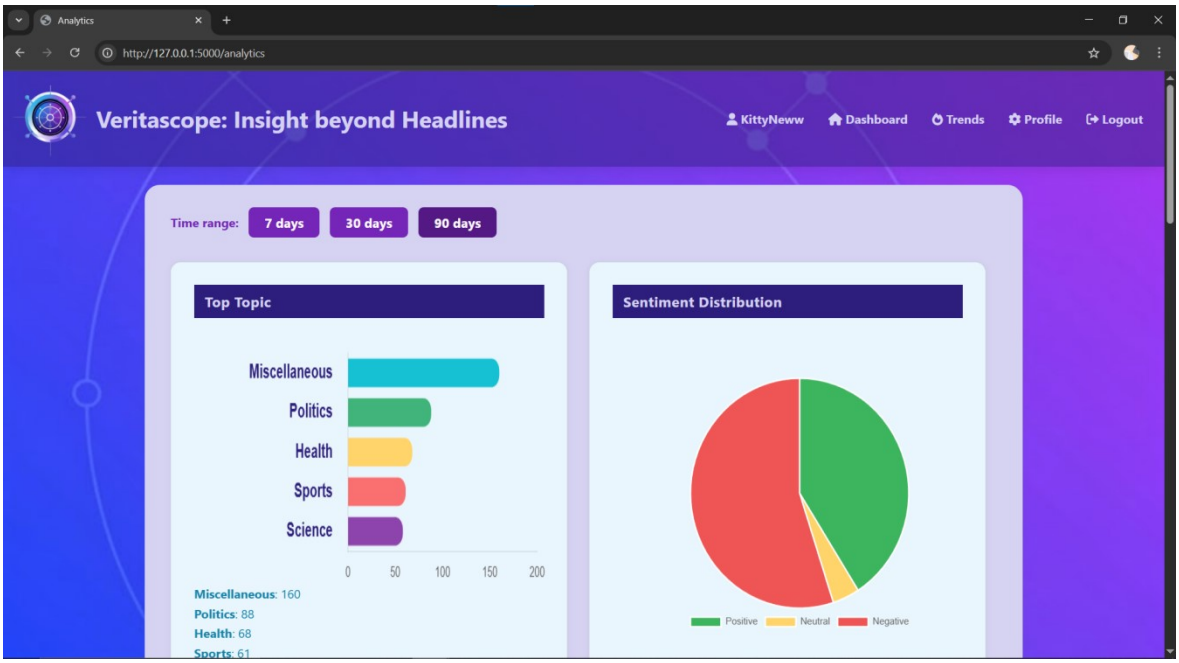
The trends page displays the top trending articles from the current week. It also highlights top keywords, topics and trending category.





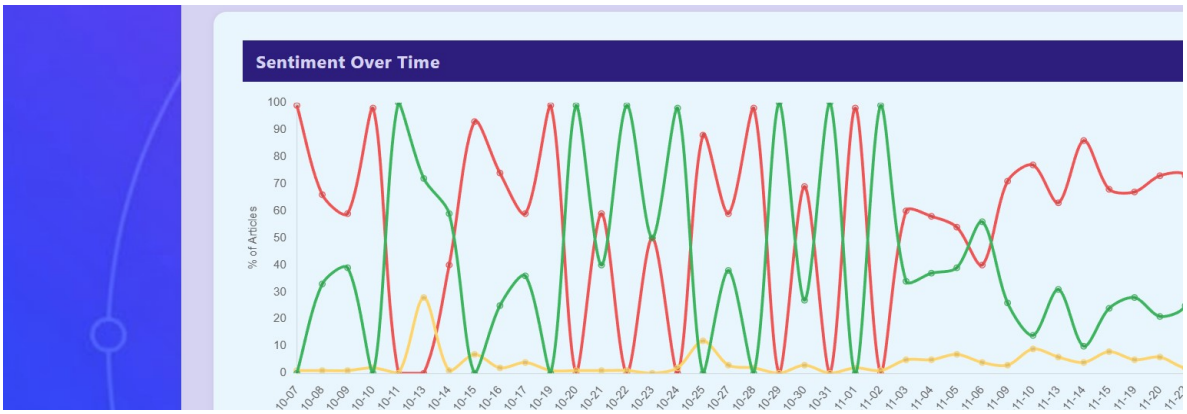
Analytics

Analytics dashboard displays the *top trending topics and sentiment distribution* for the selected timeframe (7, 30, or 90 days).

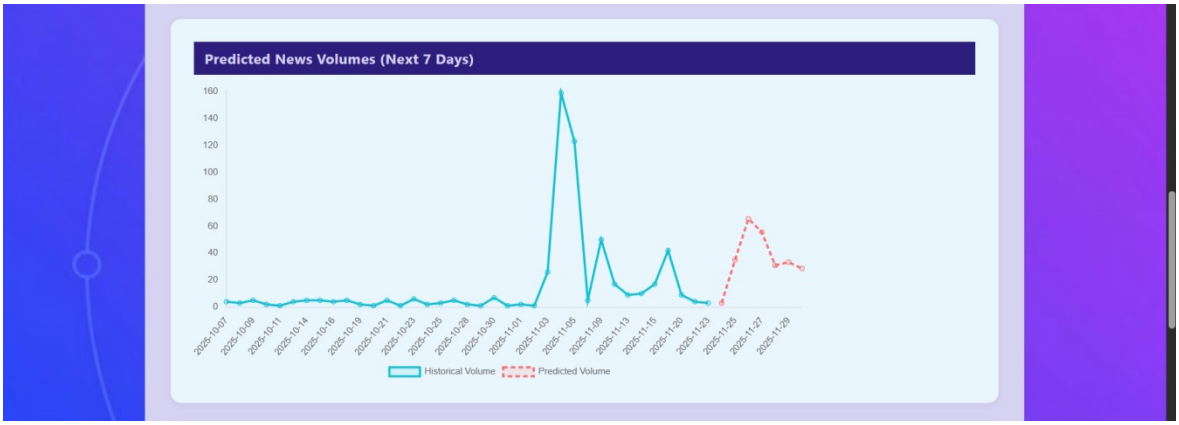


It also visualizes the evolving sentiment within news articles over a one-week period. It tracks the daily percentage of articles categorized as Positive, Neutral, or Negative, allowing for a clear analysis of trends and shifts in media tone. The visualization helps identify specific dates where

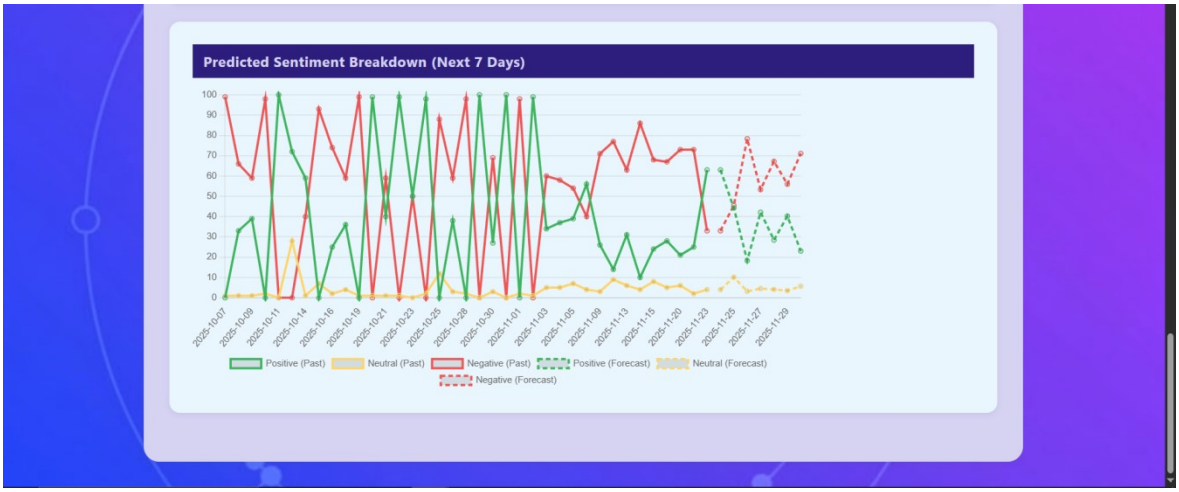
sentiment peaked or dipped significantly. This is crucial for understanding the public narrative and reacting to changes in media perception over time.



It also provides charts that forecast the expected volume of news articles, sentiment distribution and topic distribution for the upcoming seven days, based on historical data.

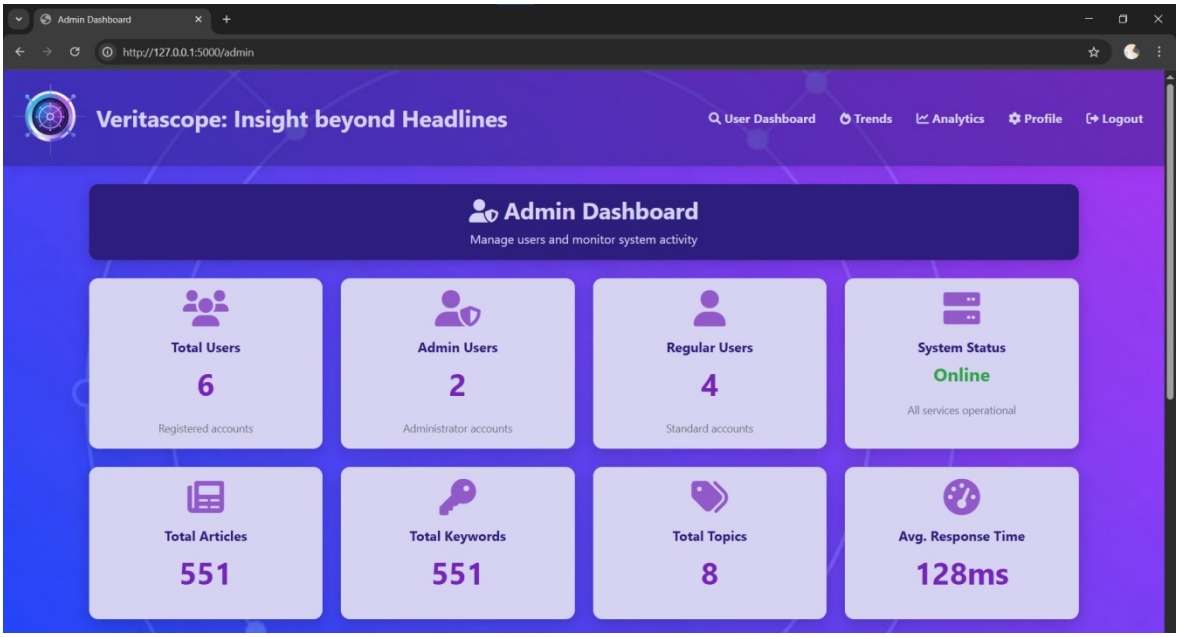


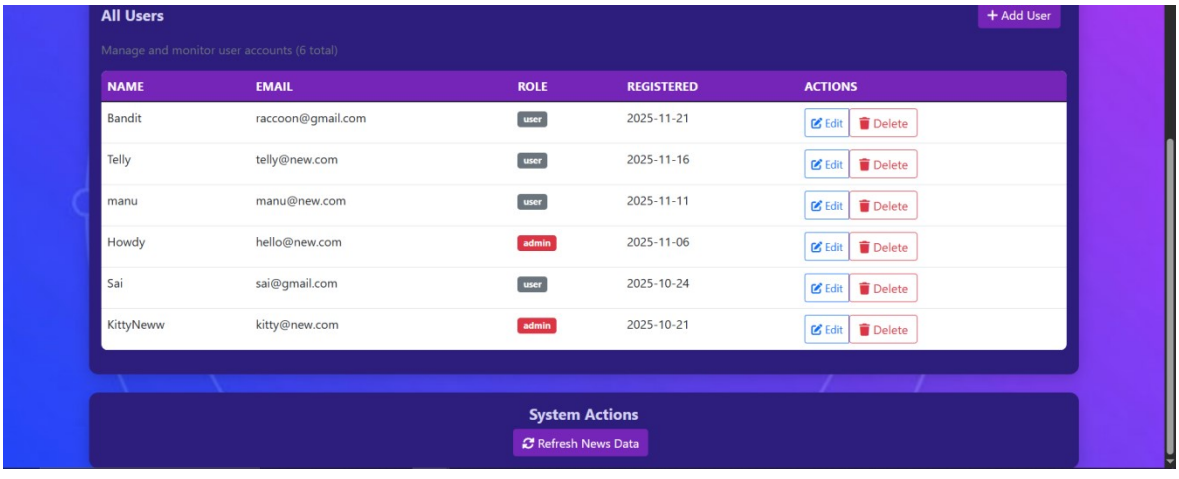




Admin Dashboard

This dashboard provides a centralized view of the platform's key metrics and user management.





8. CONCLUSION

The “Veritascope Global News Analyzer using AI” project successfully achieved its core objectives by designing, developing, and deploying an intelligent, end-to-end NLP system. The project directly addresses the challenge of information overload in the modern media landscape by providing a platform that automatically ingests, analyzes, and categorizes news articles.

The project delivered a secure, scalable, and feature-rich application. This includes a robust user authentication system using JWT , a personalized user profile module, and an automated data ingestion pipeline. The system's most significant accomplishment is its comprehensive NLP analysis suite (Module 3), which successfully integrates:

- **Keyword Extraction using KeyBERT**to provide at-a-glance summaries.
- **Sentiment Analysis using Hugging Face transformers** to determine article tone.
- **Named Entity Recognition using spaCy** to identify key people, organizations, and locations .
- **Advanced Topic Modeling using BERTopic** to cluster articles into coherent themes.

These complex data points are stored in a well-designed, relational database and presented to the user through a clean, interactive dashboard. The final platform serves as a powerful proof-of-concept, transforming unstructured news data into structured, actionable insights.

In addition, through the development of this project, the team gained valuable insights into software development in a real-world adjacent scenario. The hands-on learning, mentor guidance, peer collaboration and modular approach have contributed significantly to each team member’s experience in the Computer Science field, especially that of AI-driven application development.

9. FUTURE SCOPE

While the current Veritascope platform is a comprehensive success, its architecture is designed for significant expansion. Future development can focus on enhancing the depth of analysis, broadening the data sources, and increasing user engagement. Some of the future scope that we have identified are listed below.

- **Enhanced Personalization**
  We can create a proactive recommendation engine using the existing user_preferences and article_topics_mappingtables. This system could suggest articles and trends tailored specifically to a user's declared interests.

- **Broader Data Ingestion**
  There is a lot of scope to expand the data ingestion module to include new sources beyond the NewsAPI. Integrating data from social media APIs (like X/Twitter or Reddit) and RSS feeds would provide a more holistic view of public sentiment and emerging trends.

- **Advanced Visualizations**
  We can also enhance the "Visualization & Reporting" module by adding more interactive charts. For example, a world map visualizing topic popularity by region (using extracted GPE entities) or a line graph charting sentiment over time for a specific company or person can be added.

- **Production-Ready Scalability**
  In order to ensure smooth functioning, we can refactor the backend by containerizing the application services using Docker. This would allow us to implement a dedicated task queue for the NLP analysis functions (sentiment.py, ner.py, topic_selection.py) to run as background processes, decoupled from user requests, to ensure the web application remains fast and responsive at any scale.

10. REFERENCES

- Project Requirements Specification provided by Infosys
- Class notes provided by mentor Mrs. P. Sathya
- Flask Documentation: https://flask.palletsprojects.com/en/stable/
- Bootstrap User guide: https://getbootstrap.com/docs/5.3/getting-started/introduction/
- Infosys Springboard courses on Artificial Intelligence: https://infyspringboard.onwingspan.com/web/en/
- MySQL 8.4 Reference Manual